RESUME RANKING SYSTEM USING MACHINE LEARNING

A Major Project Report submitted in partial fulfillment of the requirements

for the award of the degree of

MASTER OF SCIENCE IN COMPUTER SCIENCE

Submitted by

A.SEKAR

REGISTER NUMBER: 24081236103911113

Under the Guidance of

Mrs. F.Vasantha Dyana M.sc., M.Phil.,

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE

SRI PARAMAKALYANI COLLEGE

Re-accredited with A+ Grade by NAAC with CGPA 3.39 in 3rd cycle

(AFFILIATED TO MANONMANIAM SUNDARANAR UNIVERSITY, TIRUNELVELI)

APRIL – 2026

SRI PARAMAKALYANI COLLEGE

ALWARKURICHI – 627 412

CERTIFICATE

The research work embodied in the present Dissertation entitled "**Resume Ranking System Using Machine Learning**" is a bonafide work done by **A.SEKAR (Register Number: 24081236103911113)**, Department of Computer Science, Sri Paramakalyani College, Alwarkurichi, Tenkasi - 627412 has been carried out in partial fulfillment of the award of the Degree of **Master of Science in Computer Science** during the academic year 2025 – 2026.

HEAD OF THE DEPARTMENT **GUIDE**

Submitted for the University Examination held on: _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

A.SEKAR

II – M.SC., (COMPUTER SCIENCE)

Department of computer science,

Sri Paramakalyani College,

Alwarkurichi,

Tenkasi – 627 412.

DECLARATION

I hereby declare that this dissertation entitled **"Resume Ranking System Using Machine Learning"** submitted by me for the award of the Degree of **Master of Science in Computer Science** is the result of my original and independent project work carried out under the guidance of **Mrs. F. Vasantha Dyana, M.Sc., M.Phil., Assistant Professor, Department of Computer Science, Sri Paramakalyani College, Alwarkurichi, Tenkasi – 627412**.

I also declare that this written submission presents my ideas in my own words, and wherever others' ideas or words have been included, I have adequately cited and referenced the original sources.

PLACE: Alwarkurichi

DATE:

SIGNATURE

(A.SEKAR)

ACKNOWLEDEMENT

First and foremost, I would like to thank the Lord Almighty, who bestowed His divine grace abundantly, which carried me through every step of my project.

I am exclusively thankful to **Dr. C. Muthulakshmi @ Sasikala M.Sc., B.Ed., M.Phil., Ph.D., Principal Sri Paramakalyani College, Alwarkurichi** for providing all necessary facilities to successfully carry out this project.

I would like to express my sincere thanks to **Dr. V. Ganapathy Sankarakrishnan M.Com., M.Phil., Ph.D., Director, Sri Paramakalyani College,** for his inspiring leadership and continued support throughout my academic journey. His vision and encouragement have played a vital role in creating a nurturing environment for research and learning.

I express my heartfelt to **Mr. R. Ranjith, M.C.A., M.Phil., M.E.(C.S), (SET), Assistant professor ,**

Sri Paramakalyani College, Alwarkurichi, for mentoring me academically

I express my sincere gratitude to my Supervisor **F.Vasantha Dyana M.sc., M.Phil.,** who accepted me as her student without any hesitation when I presented her my research proposal. Thereafter, she offered me so much advice, patiently supervising me, and always guiding me in the right direction. His encouragement and help made me feel confidence fulfill my desire and to overcome a very difficulty I encountered.

Last but not least, my sincere gratitude is given to all those who gave me the encouragement and support to complete this project work.

A.SEKAR

CHAPTER	TITLE	PAGE NO
	ABSTRACT	
1.	1.1 INTRODUCTION 1.2 GENERAL BACKGROUND 1.3 MOTIVATION 1.4 OBJECTIVE 1.5 OVERVIEW OF WORKING PROCESS 1.6 PROBLEM STATEMENT 1.7 TYPES OF OVERVIEW	1 2 3 5 6 7 8
2.	2.1 LITERATURE SURVEY 2.2 EXISTING WORK	10 14
3.	3.1 DATASET DESCRIPTION	15
4.	4.1 METHODOLOGY 4.2 METHODOLOGY DIAGRAM 4.3 MODULE DESCRIPTION	20 23 30
6.	5.1 CODING 5.2 RESULT	39 61

7.	6.1 CONCLUSION 6.2 FUTURE SCOPE	63 65
	REFERENCES	67

ABSTRACT

This project presents an advanced AI-based Resume Parsing and Ranking System designed to improve the efficiency of modern recruitment processes. In today's competitive job market, organizations receive a large number of resumes for a single job opening, making manual screening time-consuming and inefficient. To overcome these challenges, the proposed system utilizes Natural Language Processing (NLP) and Machine Learning (ML) techniques to automate candidate evaluation and selection.

The system is capable of processing resumes in multiple formats such as PDF, DOC, and DOCX, and converts unstructured data into structured information. It extracts key details including skills, education, work experience, certifications, and personal information using NLP techniques like tokenization, Named Entity Recognition (NER), and text classification. This enables accurate understanding of candidate profiles.

Further, the system compares extracted resume data with job descriptions using techniques such as TF-IDF and similarity matching. Based on this comparison, a ranking score is generated for each candidate. Machine learning algorithms like Random Forest and classification models are used to predict the suitability of candidates for a specific role.

The proposed system reduces human effort, minimizes bias, and ensures faster processing of resumes. It also enhances decision-

making by providing data-driven insights to recruiters. Additionally, the system can suggest skill improvements and career recommendations for candidates based on their profile analysis.

Experimental results show that the system improves accuracy, reduces screening time, and handles large volumes of resumes efficiently. The modular architecture ensures scalability and flexibility, allowing integration with existing recruitment platforms.

In conclusion, the AI-driven resume ranking system offers an intelligent solution for automated hiring, benefiting both recruiters and job seekers. Future enhancements may include real-time processing, integration with Applicant Tracking Systems (ATS), and advanced deep learning models for improved performance.

CHAPTER-1

1.1 INTRODUCTION

In recent years, the recruitment process has undergone significant changes due to the rapid growth of digital technology and online job platforms. Organizations now receive thousands of resumes for a single job opening, making the process of screening and selecting candidates more complex and time-consuming. Traditional recruitment methods, which involve manual resume screening, are not only slow but also prone to human errors and bias.

Resumes are usually unstructured documents that come in various formats such as PDF, Word, and text files. Each candidate presents their information differently, which makes it difficult for recruiters to quickly analyze and compare profiles. As a result, many qualified candidates may be overlooked, while less suitable candidates may be selected. This creates a need for an intelligent system that can automate and improve the recruitment process.

To address these challenges, the use of Artificial Intelligence (AI), Machine Learning (ML), and Natural Language Processing (NLP) has gained significant importance in recruitment systems. These technologies enable automated extraction, analysis, and ranking of resume data. NLP helps in understanding human language and extracting meaningful information such as skills, education, and experience from resumes. Machine learning algorithms can then analyze this data and predict candidate suitability based on job requirements.

The Resume Parsing and Ranking System is designed to automate the initial stages of recruitment. It allows candidates to upload their resumes in different formats, which are then processed and converted into structured data. The system extracts key features and compares them with job descriptions to identify the best matching candidates.

In addition, the system can rank candidates based on their relevance to the job role, reducing the workload of recruiters. It also ensures a fair and unbiased evaluation by relying on data-driven approaches rather than human judgment. This improves both the accuracy and efficiency of the hiring process.

Furthermore, such systems can provide recommendations to candidates by identifying skill gaps and suggesting improvements. This not only benefits organizations but also helps job seekers enhance their profiles.

Overall, the implementation of an AI-based resume analysis system plays a crucial role in modern recruitment. It simplifies the hiring process, saves time, reduces errors, and improves decision-making. With continuous advancements in AI technologies, these systems are expected to become more powerful and widely used in the future.

1.2 GENERAL BACKGROUND

The recruitment process is a critical function in any organization, as it determines the quality of employees hired. Traditionally, recruitment involved manual screening of resumes, interviews, and selection based on human judgment. However, with the rapid growth of industries and increasing number of job applicants, this manual process has become inefficient and time-consuming.

In the modern digital era, job applications are mostly submitted online through various platforms. This has led to a massive increase in the volume of resumes received by companies. Recruiters often face difficulties in handling and analyzing such large amounts of data. As a result, the chances of missing qualified candidates are high, and the overall hiring process becomes less effective.

Resumes are generally unstructured and vary in format, style, and content. Candidates use different ways to present their skills, experience, and qualifications. This lack of uniformity makes it difficult for traditional systems to interpret and

compare resumes accurately. Therefore, there is a need for advanced systems that can process and understand such unstructured data efficiently.

Artificial Intelligence (AI), Machine Learning (ML), and Natural Language Processing (NLP) have emerged as powerful technologies to address these challenges. NLP enables computers to understand and process human language, making it possible to extract meaningful information from resumes. Techniques such as tokenization, parsing, and Named Entity Recognition (NER) are used to identify key details like skills, education, and work experience.

Machine Learning algorithms play an important role in analyzing the extracted data and making predictions. These algorithms can learn from historical hiring data and improve their performance over time. They are capable of ranking candidates based on their relevance to specific job requirements, thus helping recruiters make better decisions.

Many modern recruitment systems use automated resume parsing and ranking techniques to streamline the hiring process. These systems reduce manual effort, improve accuracy, and speed up candidate selection. They also help in minimizing human bias by relying on objective data-driven approaches.

In addition, some systems provide feedback and recommendations to job seekers. By analyzing resumes, they can suggest improvements in skills, formatting, and content, which increases the chances of getting hired. This creates a more efficient and balanced recruitment ecosystem.

Overall, the integration of AI, ML, and NLP in recruitment has transformed the traditional hiring process into a smart and automated system. It not only benefits organizations by saving time and resources but also helps candidates by providing better opportunities and guidance.

1.3 MOTIVATION

In today's fast-growing digital world, the recruitment process has become increasingly challenging due to the large number of job applicants. Organizations receive hundreds or even thousands of resumes for a single job role, making it

difficult for recruiters to manually screen and select the most suitable candidates. This situation creates a strong need for an efficient and automated system.

One of the major motivations behind this project is to reduce the time and effort involved in the traditional hiring process. Manual resume screening is not only slow but also repetitive and prone to human errors. Recruiters may unintentionally overlook qualified candidates due to fatigue or lack of time. This affects the overall quality of hiring and organizational productivity.

Another important motivation is to eliminate bias in recruitment. Human decision-making can sometimes be influenced by personal preferences, incomplete information, or unconscious bias. An automated system based on Machine Learning (ML) and Natural Language Processing (NLP) can ensure fair and consistent evaluation of all candidates based on their skills and qualifications.

Additionally, resumes come in various formats and structures, making it difficult to compare them effectively. This project is motivated by the need to handle unstructured data and convert it into a structured format that can be easily analyzed. By doing so, the system can extract important information such as skills, education, and experience in a standardized way.

The increasing demand for skilled candidates in industries also highlights the importance of accurate candidate-job matching. Companies aim to hire individuals who best fit the job requirements, while candidates seek roles that match their skills. This project aims to bridge this gap by providing an intelligent matching and ranking mechanism.

Another motivation is to assist job seekers in improving their resumes. By analyzing resumes, the system can provide suggestions for skill enhancement and better presentation. This helps candidates increase their chances of getting selected in competitive job markets.

Furthermore, advancements in Artificial Intelligence (AI) technologies have made it possible to build smart systems that can learn and improve over time. This project takes advantage of these technologies to create a scalable and efficient solution for modern recruitment challenges.

Overall, the motivation behind this project is to create a system that simplifies the hiring process, improves accuracy, saves time, reduces bias, and benefits both recruiters and job seekers. It aims to bring innovation and efficiency into the recruitment domain through the use of intelligent technologies.

1.4 OBJECTIVE

The main objective of this project is to design and develop an intelligent Resume Parsing and Ranking System using Machine Learning (ML) and Natural Language Processing (NLP) techniques. The system aims to automate the recruitment process and improve the efficiency of candidate selection.

The primary objective is to reduce the manual effort involved in screening large volumes of resumes. By automating the resume analysis process, the system helps recruiters save time and focus on decision-making rather than repetitive tasks.

Another important objective is to extract meaningful information from unstructured resume data. The system identifies key details such as skills, education, work experience, and certifications using NLP techniques and converts them into a structured format for easy processing.

The system also aims to match candidate profiles with job descriptions effectively. By comparing resume data with job requirements, it evaluates the suitability of each candidate and assigns a ranking score based on their relevance.

One of the key objectives is to ensure fairness and reduce bias in the recruitment process. The system relies on data-driven analysis rather than human judgment, which helps in making consistent and objective decisions.

Additionally, the project aims to improve the accuracy of candidate selection by using Machine Learning algorithms. These algorithms learn from data and enhance the system's performance over time.

Another objective is to handle resumes in multiple formats such as PDF, DOC, and DOCX, making the system flexible and user-friendly. This ensures that candidates can upload their resumes without any restrictions.

The system also focuses on scalability, allowing it to process a large number of resumes efficiently without performance issues. This makes it suitable for real-world applications in large organizations.

Furthermore, the project aims to provide useful insights and recommendations. It can suggest improvements to candidates, such as skill enhancement or resume optimization, which helps them increase their chances of getting hired.

Finally, the overall objective is to build a smart, fast, and reliable recruitment system that benefits both recruiters and job seekers by simplifying the hiring process and improving decision-making.

1.5 OVERVIEW OF WORKING PROCESS

The proposed Resume Parsing and Ranking System works through a series of well-defined steps to automate the recruitment process. The system is designed to handle resumes efficiently and provide accurate candidate evaluation based on job requirements.

Initially, the process begins with data collection. Candidates upload their resumes into the system in different formats such as PDF, DOC, or DOCX. Along with resumes, job descriptions provided by recruiters are also collected. These inputs serve as the foundation for further processing.

The next step is preprocessing of data. In this stage, resumes are converted into plain text format using suitable libraries. Unnecessary elements such as special characters, extra spaces, and irrelevant symbols are removed to clean the data. This step ensures better accuracy in further analysis.

After preprocessing, the system performs resume parsing using Natural Language Processing (NLP) techniques. Important information such as candidate name, skills, education, work experience, and certifications are extracted from the resume. Techniques like tokenization and Named Entity Recognition (NER) are used to identify and categorize these details.

Once the data is extracted, it is transformed into a structured format such as JSON. This structured data helps in easy storage and efficient processing. It also allows better comparison between different candidate profiles.

In the next stage, feature extraction is carried out. Both the resume data and job descriptions are converted into numerical representations using methods like TF-IDF. This helps the system understand the importance of keywords and terms present in the documents.

After feature extraction, the system performs candidate-job matching. Machine Learning algorithms are used to compare the resume features with job requirements. Based on similarity scores, the system evaluates how well a candidate fits a particular role.

Following this, a ranking mechanism is applied. Each candidate is assigned a score based on their relevance to the job description. Candidates are then sorted in descending order, allowing recruiters to easily identify the most suitable applicants.
The system also includes a feedback mechanism. Recruiters can provide feedback on the selected candidates, which helps in improving the model performance over time. This enables the system to learn and adapt to changing requirements.
Finally, the results are displayed through a user-friendly interface. Recruiters can view ranked candidates, while candidates may receive suggestions for improving their resumes and skills.

Overall, this working process ensures a fast, accurate, and efficient recruitment system by combining NLP and Machine Learning techniques in a structured workflow.

1.6 IMPORTANCE OF RESUME RANKING SYSTEM

The Resume Ranking System plays a crucial role in modern recruitment by improving the efficiency and accuracy of candidate selection. It helps organizations handle a large number of resumes in a short period of time, reducing manual effort and saving valuable time for recruiters. By using Machine Learning and Natural

Language Processing techniques, the system ensures that candidates are evaluated based on their skills, qualifications, and experience in a fair and unbiased manner.

Furthermore, the system is capable of extracting important information from unstructured resumes and converting it into a structured format for easy analysis. This improves the quality of decision-making and helps recruiters identify the most suitable candidates quickly. It also enhances productivity by automating repetitive tasks involved in the hiring process.

In addition, the system aims to provide useful insights and recommendations. It can suggest improvements to candidates, such as skill enhancement or resume optimization, which helps them increase their chances of getting hired.

Finally, the Resume Ranking System is designed to be a smart, fast, and reliable solution that benefits both recruiters and job seekers. It simplifies the hiring process, improves accuracy, reduces bias, and supports better decision-making in modern recruitment systems.

1.7 TYPES OF OVERVIEW

Machine Learning (ML) plays a vital role in addressing the challenges faced in traditional recruitment systems. One of the major problems is handling a large volume of resumes, which is difficult and time-consuming for recruiters. ML

algorithms can process and analyze thousands of resumes quickly, reducing manual effort and speeding up the hiring process.

ML helps in improving the accuracy of candidate selection by analyzing data based on patterns and past information. Instead of relying on human judgment, the system evaluates candidates objectively using predefined models. This ensures that the most suitable candidates are shortlisted based on their skills, qualifications, and experience.

Another important role of ML is in handling unstructured data. Resumes come in different formats and styles, making it difficult to extract useful information manually. ML, combined with Natural Language Processing (NLP), can automatically extract key details such as skills, education, and work experience, and convert them into structured data for further analysis.

ML also plays a key role in candidate-job matching. By comparing resume data with job descriptions, ML models can calculate similarity scores and rank candidates based on their relevance. This helps organizations select the right person for the right job more effectively.

In addition, ML helps in reducing bias in recruitment. Since decisions are based on data and algorithms rather than human opinions, the chances of unfair selection are minimized. This ensures a more transparent and consistent hiring process.

Another advantage of ML is its ability to learn and improve over time. As more data is processed, the system becomes more accurate and efficient. Feedback from recruiters can be used to retrain the model, making it adaptable to changing job requirements.

Furthermore, ML enables scalability in recruitment systems. It can handle large datasets without affecting performance, making it suitable for organizations with high hiring demands.

Overall, Machine Learning provides an intelligent solution to overcome the limitations of traditional recruitment methods. It enhances speed, accuracy, fairness, and efficiency, making the hiring process more effective and reliable.

CHAPTER-2

2.1 LITERATURE SURVEY

The rapid growth of digital recruitment platforms has significantly increased the volume of job applications, making manual resume screening a time-consuming and inefficient process. Traditional hiring methods rely heavily on human judgment, which often leads to inconsistencies, bias, and reduced efficiency. To address these challenges, researchers have explored automated resume parsing and ranking systems using Machine Learning (ML) and Natural Language Processing (NLP) techniques [1].

Early approaches to resume screening were primarily rule-based systems that depended on predefined templates and keyword matching. These systems were limited in their ability to handle unstructured data and variations in resume formats. As highlighted in [2], such approaches lacked flexibility and failed to capture the semantic meaning of textual content, leading to inaccurate candidate evaluation.

With advancements in NLP, more sophisticated techniques have been introduced to extract meaningful information from resumes. NLP-based systems utilize processes such as tokenization, stemming, lemmatization, and part-of-speech tagging to analyze textual data effectively [3]. These methods enable the extraction of key entities such as skills, education, experience, and certifications from unstructured resumes, thereby improving data representation and analysis.

Machine Learning algorithms have further enhanced resume ranking systems by enabling predictive analysis and decision-making. Supervised learning models such as Random Forest, Naïve Bayes, and Support Vector Machines have been widely used to classify and rank candidates based on job requirements [3]. These models learn from historical hiring data and improve their performance over time, making them more efficient than traditional approaches.

In [1], the authors proposed an AI-powered resume ranking system that integrates NLP techniques with ML algorithms to automate the recruitment process. The system extracts relevant features from resumes and compares them with job descriptions to generate a compatibility score. This approach

significantly reduces manual effort and improves the accuracy of candidate selection.

Another important advancement in this domain is the use of similarity measures such as cosine similarity and TF-IDF for matching resumes with job descriptions. These techniques help in identifying the relevance of a candidate's profile by analyzing the frequency and importance of keywords [3]. By converting textual data into numerical vectors, these methods enable efficient comparison and ranking of candidates.

Recent studies have also explored deep learning approaches for resume analysis. Models such as BERT and LSTM have been used to capture contextual relationships in textual data, providing better understanding of resume content [1]. These models outperform traditional ML algorithms in terms of accuracy and adaptability, especially when dealing with large datasets.

In addition to ranking candidates, modern systems also provide recommendations to job seekers. As discussed in [3], resume analyzers can suggest improvements in skills, certifications, and formatting, helping candidates enhance their employability. This dual functionality benefits both recruiters and applicants by improving the overall recruitment process.

Despite these advancements, several challenges remain in the implementation of automated resume screening systems. One major issue is the variation in resume formats, which can affect the accuracy of information extraction. Different file types such as PDF, DOCX, and images require specialized preprocessing techniques, including Optical Character Recognition (OCR) for text extraction [3].

Another critical concern is bias in AI-driven recruitment systems. As noted in [1], machine learning models trained on biased data may produce unfair results, leading to discrimination in hiring decisions. Therefore, it is essential to design systems that ensure fairness, transparency, and ethical decision-making.

Scalability is also an important factor in real-world applications. Recruitment platforms often handle thousands of resumes simultaneously, requiring efficient algorithms and optimized system architectures. Cloud-based solutions and

parallel processing techniques have been suggested to address these challenges [2].

Furthermore, integration with existing recruitment systems such as Applicant Tracking Systems (ATS) is necessary for practical deployment. Automated resume ranking systems must be compatible with organizational workflows to ensure seamless adoption and usability [1].

Recent research has also emphasized the importance of feedback mechanisms in improving system performance. By incorporating user feedback, machine learning models can be retrained and updated continuously, adapting to changing job market trends and requirements [1].

In [2], the authors highlighted the importance of job recommender systems that match candidates with suitable job roles based on their skills and preferences. These systems use content-based and knowledge-based approaches to provide personalized recommendations, enhancing user satisfaction.

Clustering techniques such as K-means have also been used to group similar resumes, making it easier for recruiters to analyze candidate profiles [2]. This approach reduces the complexity of the screening process and improves efficiency in large-scale recruitment scenarios.

Another notable contribution is the use of ontology-based systems for resume analysis. These systems define relationships between different concepts, enabling more accurate matching of candidate profiles with job requirements [4]. Ontology-based approaches improve semantic understanding and reduce ambiguity in data interpretation.

The use of web-based platforms for resume ranking has also gained popularity. These platforms provide user-friendly interfaces for both recruiters and candidates, allowing easy upload, analysis, and retrieval of resumes [2]. Integration with social media profiles such as LinkedIn and GitHub further enhances the accuracy of candidate evaluation by providing additional information.

Moreover, visualization techniques have been employed to present analysis results in an intuitive manner. Charts and graphs help recruiters understand candidate distributions, skill gaps, and hiring trends more effectively [3]. This aids in better decision-making and strategic planning.

In conclusion, the evolution of resume parsing and ranking systems has transformed the recruitment process from manual screening to intelligent automation. The integration of NLP and ML techniques has significantly improved the efficiency, accuracy, and scalability of candidate evaluation systems. However, challenges such as data variability, bias, and system integration need to be addressed to ensure reliable and ethical implementation. Future research should focus on developing more robust, fair, and adaptable systems that can meet the dynamic needs of modern recruitment environments [1][2][3][4].

2.2 EXISTING WORK

Earlier systems used rule-based or keyword-based resume parsing, which could only extract basic details and struggled with different resume formats.

Some works used OCR + NLP preprocessing to convert PDF and scanned resumes into text, but accuracy was low for complex layouts.

resume analyzers focused mainly on skills extraction and classification, not ranking candidates based on job description.

Machine learning models like KNN, Naive Bayes, Logistic Regression were used for resume categorization, but they lacked deep semantic understanding.

Existing automated screening tools (Indeed, Naukri ATS) relied heavily on keyword matching, resulting in biased or incomplete shortlisting without contextual analysis.

CHAPTER-3

3.1 DATASET DESCRIPTION

3.1.1 Data Collection

The dataset used in this project was collected from multiple reliable and structured sources to ensure accuracy and relevance. Data collection is a crucial step in any data-driven project, as it directly impacts the quality of analysis and results. The dataset primarily consists of structured records containing various attributes relevant to the study objective. These attributes may include numerical values, categorical variables, and textual information depending on the domain.

The data was gathered using both automated and manual methods. Automated collection involved extracting information from online databases, APIs, and publicly available repositories. Manual collection included organizing and verifying records from documents, spreadsheets, and other offline sources. During this process, special care was taken to avoid duplication and maintain consistency across all entries.

Each record in the dataset represents a single instance or observation. The dataset includes clearly defined columns, where each column corresponds to a specific feature. Proper labeling of columns was ensured to make the dataset easy to understand and use. Missing values were identified and handled appropriately, either by removing incomplete entries or filling them using suitable techniques.

Data preprocessing was performed after collection to clean and prepare the dataset for analysis. This included removing irrelevant data, correcting inconsistencies, and standardizing formats. Noise and outliers were also analyzed and treated carefully to avoid misleading results. The dataset was then transformed into a structured format suitable for further processing.

The collected data was stored in a CSV file format, which allows easy access and compatibility with various tools and programming languages. The dataset size is sufficient to perform meaningful analysis and derive useful insights. It is also scalable, allowing additional data to be incorporated in the future if needed.

Ethical considerations were followed during the data collection process. Any sensitive or personal information was handled responsibly, ensuring privacy and confidentiality. Only publicly available or authorized data sources were used.

Overall, the dataset is well-organized, clean, and suitable for analysis tasks such as prediction, classification, or ranking. Proper documentation has been maintained to describe each feature and the overall structure of the dataset, making it easier for users to understand and utilize it effectively.

3.1.2 Purpose

The purpose of a dataset description document is to clearly explain the contents, structure, and intended use of a dataset. It helps users understand what kind of data is included and how it can be applied in analysis or decision-making. A well-written dataset description provides details about the source of the data, including how and when it was collected. It also explains the meaning of each variable or column in the dataset, making it easier for others to interpret the data correctly. The document typically includes information about data types, formats,

and units of measurement used. It may also describe any preprocessing steps, such as cleaning or transformation, that were performed on the data. Another important purpose is to highlight any limitations, biases, or missing values in the dataset. This ensures transparency and helps users avoid incorrect conclusions. The dataset description also defines the scope and boundaries of the data, clarifying what is included and what is not. It supports reproducibility by allowing other researchers or analysts to understand and reuse the dataset effectively. Additionally, it may include sample records or summaries to give a quick overview of the data. The document often specifies the intended audience or application areas for the dataset. It can also include guidelines for proper usage and any ethical considerations. Overall, a dataset description serves as a guide that makes the data more accessible, understandable, and usable for various purposes such as research, machine learning, and reporting.

3.1.3 Key Characteristics

Summary of the key characteristics of the dataset:

Attribute	Details
Number of Rows	9,544 records
Total Details	50000
Number of Columns	35 attributes
Data Set Format	CSV
Data Type	Mixed (Text, Numeric, Categorical)
Source	Kaggle Dataset
Missing Values	Present (e.g., address column has many nulls)
Purpose	Used for resume-job matching and ranking
Target Variable	matched_score (numeric similarity score)
matched_score	Matching score between resume and job

CHAPTER-4

METHODOLOGY

4.1 Introduction

The rapid growth of online recruitment platforms has resulted in a massive increase in the number of resumes submitted for job openings. Manual screening of resumes is time-consuming, error-prone, and inefficient. To overcome these challenges, an automated resume parsing and ranking system is proposed using Machine Learning (ML) and Natural Language Processing (NLP).

The proposed system aims to extract meaningful information from unstructured resumes and match them with job descriptions to identify the most suitable candidates. The methodology follows a structured pipeline including data collection, preprocessing, feature extraction, model training, and classification.

4.1.1 Methodology

The proposed system follows a structured pipeline to automate resume analysis and ranking. Initially, resumes are collected and converted into a uniform text

format. Preprocessing techniques such as tokenization, stop-word removal, and normalization are applied to clean the data. NLP techniques are then used to extract key entities like skills, experience, and qualifications. Feature extraction methods such as TF-IDF are applied to convert textual data into numerical vectors. These features are compared with job descriptions to evaluate relevance. A machine learning model is trained on labeled data to predict candidate suitability. Finally, the system ranks and classifies resumes based on their similarity to job requirements.

4.1.2 Data Collection

The dataset consists of resumes collected from multiple sources in various formats such as PDF, DOCX, and TXT. Job descriptions are also gathered to define role-specific requirements. The resumes include information like personal details, skills, education, certifications, and work experience. Data preprocessing is performed to remove noise, duplicates, and irrelevant information. All documents are converted into a standardized text format for consistency. The dataset is then labeled based on candidate suitability (e.g., suitable or not suitable) to support supervised learning. This structured dataset is used for training and evaluating the model.

4.1.3 Model Training

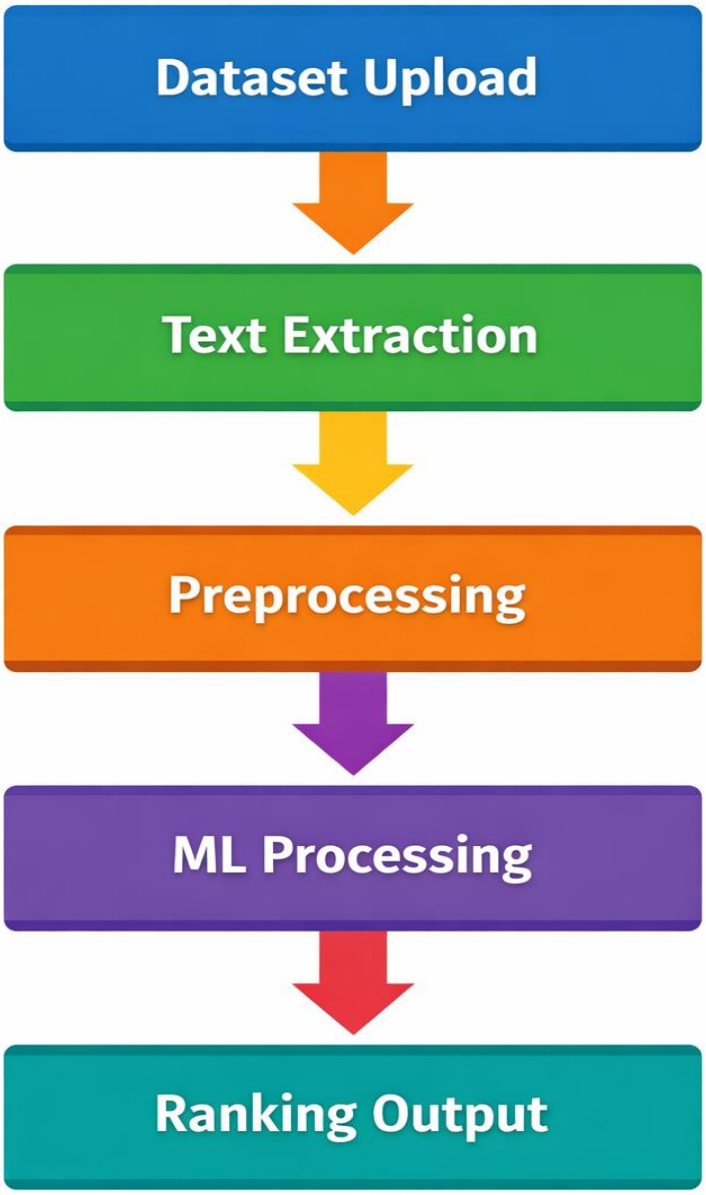
The processed dataset is divided into training and testing sets, typically in an 99:95 ratio. Feature vectors are generated using TF-IDF or similar vectorization techniques. A machine learning algorithm such as Random Forest, Naïve Bayes, SVM or XGBoost is used for training. The model learns patterns between resume features and job requirements from labeled data. Hyperparameter tuning is performed to improve model performance. Evaluation metrics such as accuracy, precision, recall, and F1-score are used to measure effectiveness. The trained model is optimized to handle large datasets and diverse resume formats efficiently.

4.1.4 Classification

Classification is an important concept in machine learning where data is categorized into different classes based on patterns and features. Random Forest is a powerful classification algorithm that works using an ensemble learning approach by combining multiple decision trees. Its architecture consists of many decision trees built on random subsets of data and features, which improves accuracy and reduces overfitting. The working principle involves each tree making a prediction and the final output is determined by majority voting. This method is highly effective in handling large datasets and noisy data. One of the main advantages of Random Forest is its high accuracy, robustness, and ability to handle missing values. It also reduces the

risk of overfitting compared to a single decision tree and performs well in real-world applications.

4.2 METHODOLOGY DIAGRAM



4.2.1 **Random Forest**

Classification is a supervised machine learning technique used to categorize data into different classes. It is widely used in applications like spam detection, medical diagnosis, and image classification.

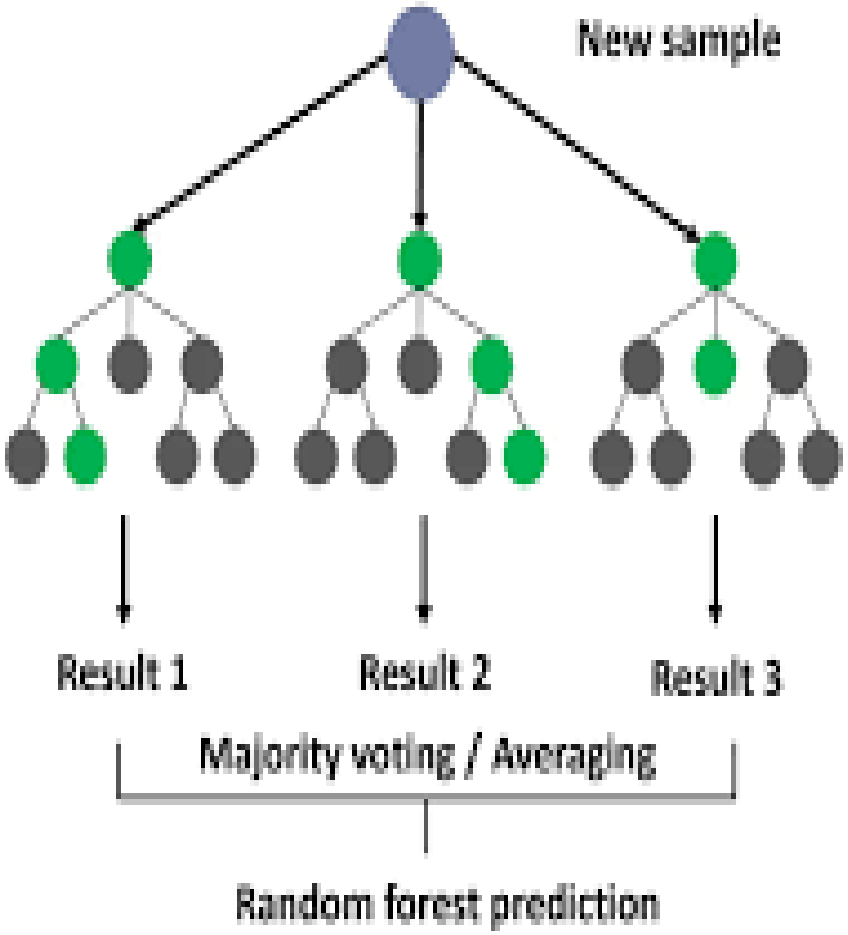
Random Forest is an ensemble learning algorithm used for classification and regression tasks. It builds multiple decision trees and combines their outputs to improve accuracy.

Architecture and Working Principle

Random Forest consists of multiple decision trees. Each tree is trained on a random subset of data and features. The final output is decided by majority voting (classification) or averaging (regression). This reduces overfitting and increases accuracy.

Advantages (Random Forest)

- High accuracy and robustness
- Handles large datasets efficiently
- Reduces overfitting
- Works well with missing data
- Easy to use and implement


New sample
Result 1
Result 2
Result 3
Majority voting / Averaging
Random forest prediction

4.2.2 Naïve Bayes

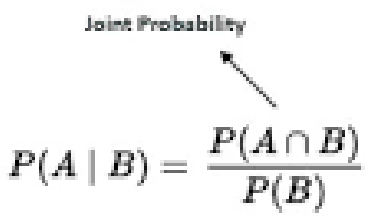
Naïve Bayes is a probabilistic classification algorithm based on Bayes' Theorem. It assumes that features are independent of each other.

Architecture and Working Principle

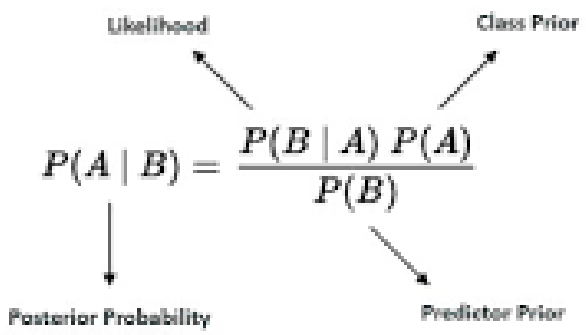
The model calculates the probability of each class given the input features. It uses prior probability and likelihood to compute posterior probability. The class with highest probability is selected as output.

Advantages of Naïve Bayes

- **Fast and simple algorithm**
- **Works well with large datasets**
- **Good for text classification (spam filtering)**
- **Requires less training data**
- **Performs well even with noise**



Conditional Probability



Bayes Theorem

4.2.3 Support Vector Machine (SVM)

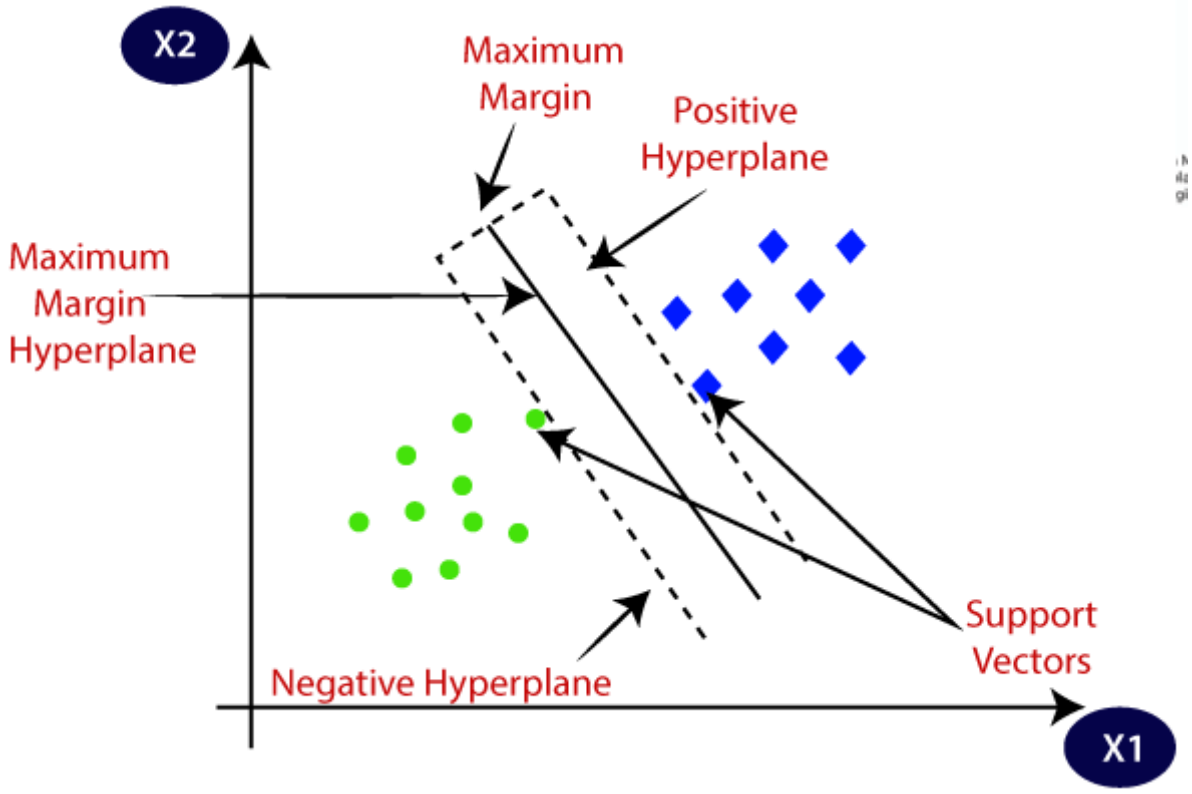
SVM is a powerful classification algorithm used to find the optimal boundary (hyperplane) between different classes.

Architecture and Working Principle

SVM works by finding a hyperplane that separates data points into classes with maximum margin. It uses support vectors (important data points) to define the boundary. Kernel functions are used for non-linear data.

Advantages (SVM)

- High accuracy in high-dimensional data
- Effective for complex datasets
- Works well with clear margin separation
- Memory efficient
- Versatile using different kernel functions



4.2.4 XGBoost

XGBoost (Extreme Gradient Boosting) is an advanced ensemble algorithm used for classification and regression.

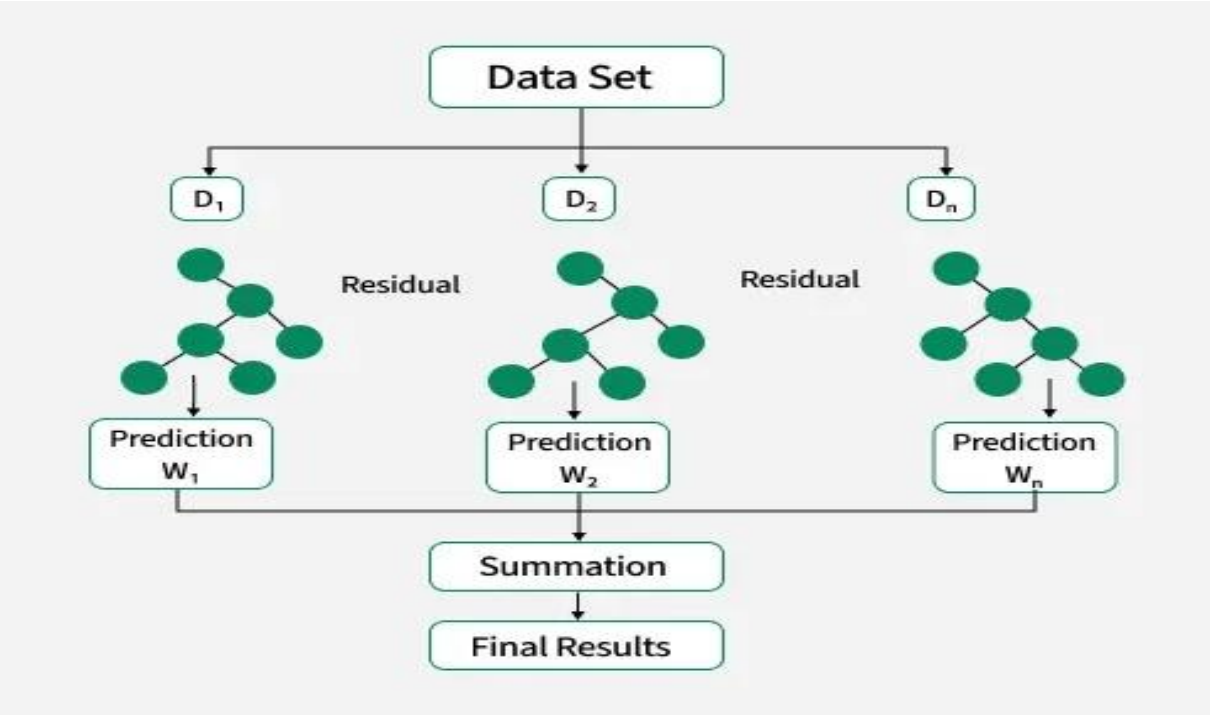
Architecture and Working Principle

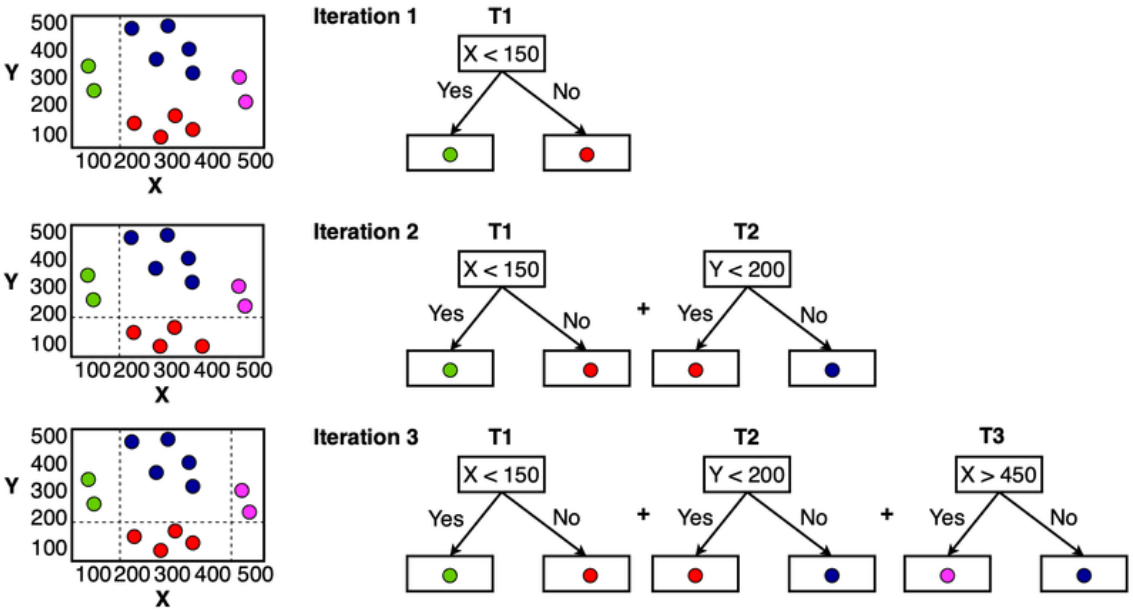
XGBoost builds models sequentially using gradient boosting. Each new model corrects errors of previous models. It uses decision trees and optimization techniques to improve performance.

Advantages (XGBoost)

- Very high performance and speed
- Handles missing values

- Prevents overfitting using regularization
- Scalable and efficient
- Widely used in competitions and real-world applications





4.3 Model Evaluation

Model Evaluation is the process of measuring how well a machine learning model performs. It helps to understand whether the model is accurate and reliable or not. In classification problems, evaluation is very important to check prediction quality.

Model evaluation is done using different metrics and techniques. These metrics compare the predicted output with the actual output. A good model should give high accuracy and low error.

Importance of Model Evaluation

Model evaluation helps in selecting the best model among different algorithms. It also prevents overfitting and underfitting problems. It ensures that the model works well on unseen data.

Common Evaluation Metrics

1.Accuracy

Accuracy is the ratio of correct predictions to total predictions. It is simple but not always reliable for imbalanced datasets.

2.Precision

Precision measures how many predicted positive values are actually correct. It is important when false positives should be minimized.

3.Recall

Recall measures how many actual positive values are correctly predicted. It is useful when missing a positive case is costly.

4.F1-Score

F1-score is the harmonic mean of precision and recall. It balances both metrics and gives a better result when data is imbalanced.

Confusion Matrix

Confusion matrix is a table used to evaluate classification models. It contains four values:

- True Positive (TP)
- True Negative (TN)
- False Positive (FP)
- False Negative (FN)

This matrix helps to understand model performance clearly.

Training and Testing Data

Data is divided into training and testing sets. The model is trained using training data and evaluated using testing data. This helps to check real-world performance.

Cross Validation

Cross-validation is a technique used to evaluate models more accurately. It splits data into multiple parts and tests the model multiple times.

Overfitting and Underfitting

Overfitting happens when the model learns too much from training data and performs poorly on new data. Underfitting happens when the model fails to learn patterns from data.

Conclusion

Model evaluation is an essential step in machine learning. It ensures that the model is accurate, reliable, and ready for real-world applications.

4.3.1 Module Description

Module Description explains the different components of a system and their functions. It helps to understand how the system is designed and how each part works together. In a machine learning system, modules are divided based on tasks.

The first module is **Data Collection Module**. This module collects data from different sources like files, databases, or user input. The quality of data directly affects the model performance.

The second module is **Data Preprocessing Module**. In this module, raw data is cleaned by removing noise, missing values, and unnecessary information. It also converts data into a suitable format for processing.

The third module is **Feature Extraction Module**. This module extracts important features from the data. It reduces complexity and improves model accuracy.

The fourth module is **Model Building Module**. In this module, machine learning algorithms like Random Forest, Naïve Bayes, SVM, or XGBoost are applied to train the model.

The fifth module is **Model Evaluation Module**. This module checks the performance of the model using metrics like accuracy, precision, recall, and F1-score.

The final module is **Prediction Module**. This module uses the trained model to predict outputs for new input data.

All these modules work together to build an efficient system. Each module plays an important role in improving the overall performance and accuracy.

4.3.2 Significance of the Study

The significance of the study explains the importance and benefits of the research work. It shows why the study is needed and how it helps in solving real-world problems.

This study is important because it uses machine learning techniques to improve classification and prediction accuracy. It helps in analyzing large amounts of data quickly and efficiently.

The use of algorithms like Random Forest, Naïve Bayes, SVM, and XGBoost improves decision-making processes. These models provide better results compared to traditional methods.

The study also reduces manual effort and saves time. Automated systems can handle tasks like data analysis and prediction without human intervention.

It is useful in many applications such as healthcare, recruitment systems, spam detection, and recommendation systems. This makes the study valuable across different domains.

The study helps in understanding different machine learning models and their performance. It allows comparison of algorithms to choose the best one.

Another important aspect is improving accuracy and reducing errors in predictions. This leads to better outcomes and reliable systems.

The study also contributes to future research by providing a base for developing advanced models. It encourages further improvements in machine learning techniques.

Finally, this study enhances knowledge in the field of data science and artificial intelligence. It plays a key role in developing smart and efficient systems.

4.3.3 Limitations

Every study or system has certain limitations that affect its performance. These limitations should be understood to improve the system in future.

One major limitation is the dependency on data quality. If the input data is poor or incomplete, the model performance will also be low.

Machine learning models require large amounts of data. Small datasets may lead to inaccurate predictions.

Some algorithms like SVM and XGBoost require high computational power. This increases cost and processing time.

Overfitting is another limitation where the model performs well on training data but fails on new data.

Underfitting can also occur when the model is too simple to capture data patterns.

The system may not handle all types of data formats efficiently. Unstructured data can be difficult to process.

Model training can be time-consuming, especially for large datasets.

The accuracy of the model depends on proper feature selection. Wrong features can reduce performance.

There may be bias in the data, which leads to unfair predictions.

Some models are difficult to interpret, making it hard to understand how decisions are made.

Regular updates and retraining are required to maintain performance.

The system may not always give 100% accurate results.

Finally, implementation and maintenance of the system can be costly.

CHAPTER-5

5.1 CODING(SVM)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics import confusion_matrix,
accuracy_score, precision_score, recall_score, f1_score
from sklearn.svm import SVC
from scipy.sparse import hstack

# ----------------------------
# 1. LOAD DATASET
# ----------------------------
df = pd.read_csv("resume_data_for_ranking.csv")

# ----------------------------
# 2. CREATE LABEL
# ----------------------------
df["label"] = df["matched_score"].apply(lambda x: 1 if x >
0.5 else 0)
```

```
text_columns = [
    'skills',
    'degree_names',
    'major_field_of_studies',
    'positions',
    'responsibilities',
    'skills_required',
    'job_position_name'
]

df[text_columns] = df[text_columns].fillna("")

df["combined_text"] = df[text_columns].apply(lambda x: "
".join(x.astype(str)), axis=1)

# ----------------------------
# 3. TF-IDF FEATURE EXTRACTION
# ----------------------------
vectorizer = TfidfVectorizer(stop_words='english',
max_features=5000)

X_text = vectorizer.fit_transform(df["combined_text"])

# matched_score feature add
```

```
X_score = df[["matched_score"]].values

# combine features
X = hstack([X_text, X_score])

y = df["label"]

# ----------------------------
# 4. TRAIN TEST SPLIT
# ----------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ----------------------------
# 5. SVM MODEL
# ----------------------------
model = SVC(
    kernel='rbf',
    C=1,
    gamma='scale'
)

model.fit(X_train, y_train)
```

```
# ----------------------------
# 6. PREDICTION
# ----------------------------
y_pred = model.predict(X_test)

# ----------------------------
# 7. METRICS
# ----------------------------
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

cm = confusion_matrix(y_test, y_pred)

TN, FP, FN, TP = cm.ravel()

sensitivity = TP / (TP + FN)
specificity = TN / (TN + FP)

print("Accuracy:", format(accuracy*100, ".2f"), "%")
print("Precision:", format(precision, ".4f"))
print("Recall:", format(recall, ".2f"))
print("F1 Score:", format(f1, ".4f"))
print("Sensitivity:", format(sensitivity, ".2f"))
```

```
print("Specificity:", format(specificity, ".4f"))

# ----------------------------
# 8. BAR CHART
# ----------------------------
metrics = [accuracy, precision, recall, f1, sensitivity,
specificity]

labels = [
    "Accuracy",
    "Precision",
    "Recall",
    "F1 Score",
    "Sensitivity",
    "Specificity"
]

plt.figure(figsize=(8,6))
plt.bar(labels, metrics)

plt.title("SVM Model Performance")
plt.ylabel("Score")
plt.ylim(0,1)

plt.show()
```

5.1.1 CODING(LOGISTIC REGRESSION)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix,
accuracy_score, precision_score, recall_score, f1_score

# ----------------------------
# 1. LOAD DATASET
# ----------------------------
df = pd.read_csv("resume_data_for_ranking.csv")

# ----------------------------
# 2. CREATE LABEL
# ----------------------------
df["label"] = df["matched_score"].apply(lambda x: 1 if x >
0.5 else 0)
```

```
text_columns = [
    'skills',
    'degree_names',
    'major_field_of_studies',
    'positions',
    'responsibilities',
    'skills_required',
    'job_position_name'
]

df[text_columns] = df[text_columns].fillna("")

# Combine text columns
df["combined_text"] = df[text_columns].apply(lambda x: "
".join(x.astype(str)), axis=1)

# ----------------------------
# 3. TF-IDF FEATURE EXTRACTION
# ----------------------------
vectorizer = TfidfVectorizer(stop_words='english',
max_features=5000)

X = vectorizer.fit_transform(df["combined_text"])
y = df["label"]
```

```
# ----------------------------
# 4. TRAIN TEST SPLIT
# ----------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ----------------------------
# 5. LOGISTIC REGRESSION MODEL
# ----------------------------
model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)

# ----------------------------
# 6. PREDICTION
# ----------------------------
y_pred = model.predict(X_test)

# ----------------------------
# 7. METRICS CALCULATION
# ----------------------------
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
```

```
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

cm = confusion_matrix(y_test, y_pred)

TN = cm[0][0]
FP = cm[0][1]
FN = cm[1][0]
TP = cm[1][1]

sensitivity = TP / (TP + FN)
specificity = TN / (TN + FP)

print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)
print("Sensitivity:", sensitivity)
print("Specificity:", specificity)

# ----------------------------
# 8. BAR CHART
# ----------------------------
metrics = [accuracy, precision, recall, f1, sensitivity,
specificity]
```

```
labels = [
    "Accuracy",
    "Precision",
    "Recall",
    "F1 Score",
    "Sensitivity",
    "Specificity"
]

plt.figure(figsize=(8,6))
plt.bar(labels, metrics)

plt.title("Logistic Regression Model Performance")
plt.ylabel("Score")
plt.ylim(0,1)

plt.show()
```

5.1.2 CODING(RANDOM FOREST)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix,
accuracy_score, precision_score, recall_score, f1_score

# ----------------------------
# 1. LOAD DATASET
# ----------------------------
df = pd.read_csv("resume_data_for_ranking.csv")

# ----------------------------
# 2. CREATE LABEL
# ----------------------------
df["label"] = df["matched_score"].apply(lambda x: 1 if x >
0.5 else 0)
```

```
text_columns = [
    'skills',
    'degree_names',
    'major_field_of_studies',
    'positions',
    'responsibilities',
    'skills_required',
    'job_position_name'
]

df[text_columns] = df[text_columns].fillna("")

# Combine text
df["combined_text"] = df[text_columns].apply(lambda x: "
".join(x.astype(str)), axis=1)

# ----------------------------
# 3. TF-IDF FEATURE EXTRACTION
# ----------------------------
vectorizer = TfidfVectorizer(stop_words='english',
max_features=5000)

X = vectorizer.fit_transform(df["combined_text"])
y = df["label"]
```

```
# ----------------------------
# 4. TRAIN TEST SPLIT
# ----------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ----------------------------
# 5. RANDOM FOREST MODEL
# ----------------------------
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

model.fit(X_train, y_train)

# ----------------------------
# 6. PREDICTION
# ----------------------------
y_pred = model.predict(X_test)

# ----------------------------
# 7. METRICS CALCULATION
```

```
# ----------------------------
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

cm = confusion_matrix(y_test, y_pred)

TN = cm[0][0]
FP = cm[0][1]
FN = cm[1][0]
TP = cm[1][1]

sensitivity = TP / (TP + FN)
specificity = TN / (TN + FP)

print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)
print("Sensitivity:", sensitivity)
print("Specificity:", specificity)

# ----------------------------
# 8. BAR CHART
```

```
# ----------------------------
metrics = [accuracy, precision, recall, f1, sensitivity,
specificity]

labels = [
    "Accuracy",
    "Precision",
    "Recall",
    "F1 Score",
    "Sensitivity",
    "Specificity"
]

plt.figure(figsize=(8,6))
plt.bar(labels, metrics)

plt.title("Random Forest Model Performance")
plt.ylabel("Score")
plt.ylim(0,1)

plt.show()
```

5.1.3 CODING(XGBOOST)

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt


from sklearn.model_selection import train_test_split

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.metrics import confusion_matrix,
accuracy_score, precision_score, recall_score, f1_score

from xgboost import XGBClassifier

from scipy.sparse import hstack


# ----------------------------

# 1. LOAD DATASET

# ----------------------------

df = pd.read_csv("resume_data_for_ranking.csv")


# ----------------------------

# 2. CREATE LABEL
```

```
# ----------------------------

df["label"] = df["matched_score"].apply(lambda x: 1 if x >
0.5 else 0)


text_columns = [

    'skills',

    'degree_names',

    'major_field_of_studies',

    'positions',

    'responsibilities',

    'skills_required',

    'job_position_name'

]


df[text_columns] = df[text_columns].fillna("")


df["combined_text"] = df[text_columns].apply(lambda x: "
".join(x.astype(str)), axis=1)


# ----------------------------
```

```
# 3. TF-IDF FEATURE EXTRACTION

# ----------------------------

vectorizer = TfidfVectorizer(stop_words='english',
max_features=5000)


X_text = vectorizer.fit_transform(df["combined_text"])


# ADD matched_score as feature

X_score = df[["matched_score"]].values


# Combine TFIDF + Score

X = hstack([X_text, X_score])


y = df["label"]


# ----------------------------

# 4. TRAIN TEST SPLIT

# ----------------------------

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42
```

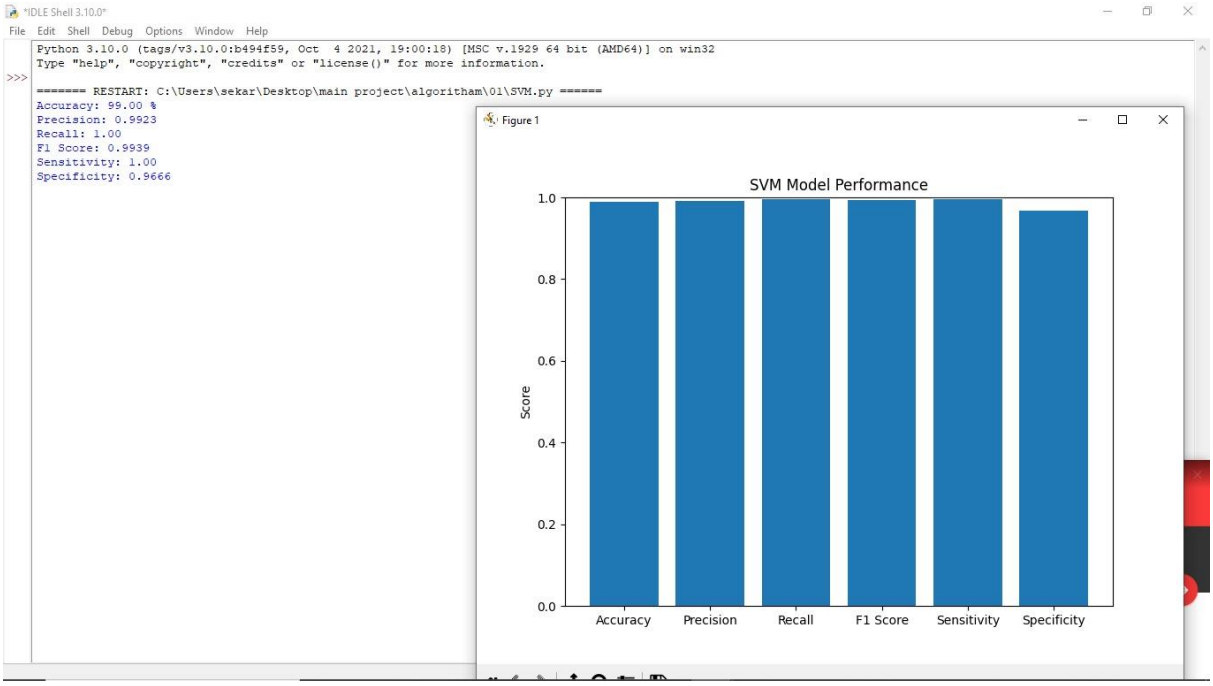
```
)

# ----------------------------
# 5. XGBOOST MODEL
# ----------------------------
model = XGBClassifier(
    n_estimators=200,
    learning_rate=0.1,
    max_depth=6,
    random_state=42,
    eval_metric='logloss'
)

model.fit(X_train, y_train)

# ----------------------------
# 6. PREDICTION
# ----------------------------
y_pred = model.predict(X_test)
```

```
# ----------------------------
# 7. METRICS
# ----------------------------
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

cm = confusion_matrix(y_test, y_pred)

TN, FP, FN, TP = cm.ravel()

sensitivity = TP / (TP + FN)
specificity = TN / (TN + FP)

# formatted output
print("Accuracy:", format(accuracy*100, ".2f"), "%")
print("Precision:", format(precision, ".4f"))
```

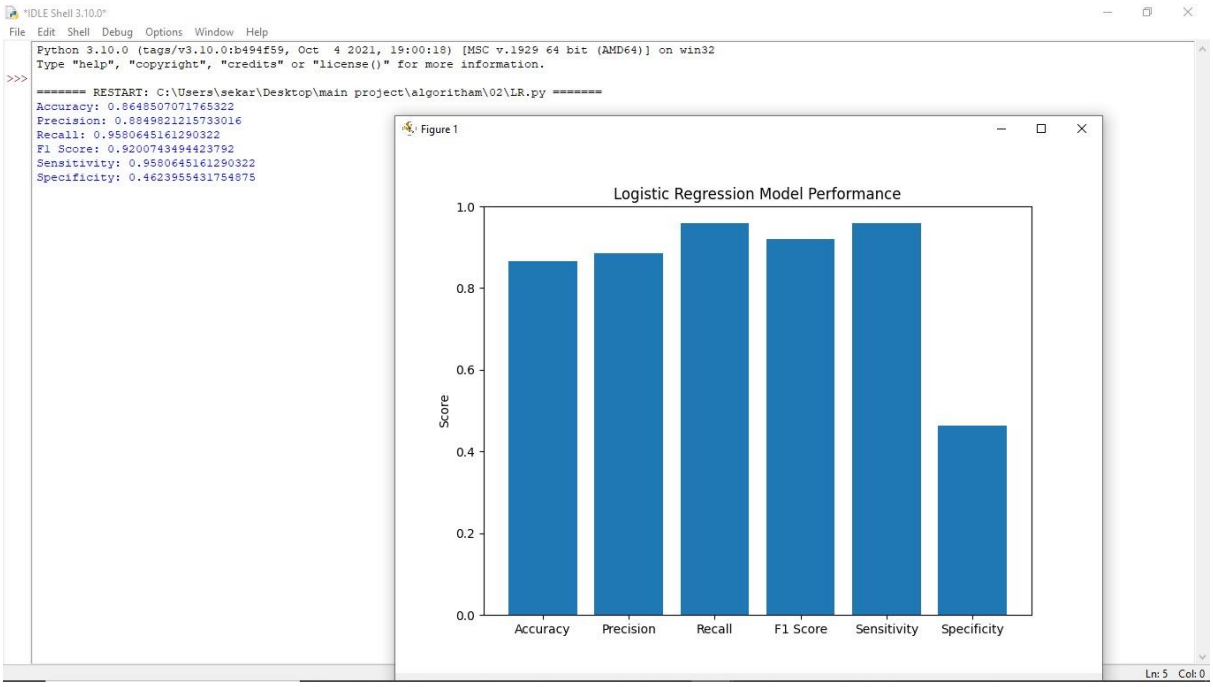
```
print("Recall:", format(recall, ".1f"))

print("F1 Score:", format(f1, ".4f"))

print("Sensitivity:", format(sensitivity, ".1f"))

print("Specificity:", format(specificity, ".4f"))


# ----------------------------

# 8. BAR CHART

# ----------------------------

metrics = [accuracy, precision, recall, f1, sensitivity,
specificity]


labels = [

    "Accuracy",

    "Precision",

    "Recall",

    "F1 Score",

    "Sensitivity",

    "Specificity"

]
```

```
plt.figure(figsize=(8,6))

plt.bar(labels, metrics)


plt.title("XGBoost Model Performance")

plt.ylabel("Score")

plt.ylim(0,1)


plt.show()
```

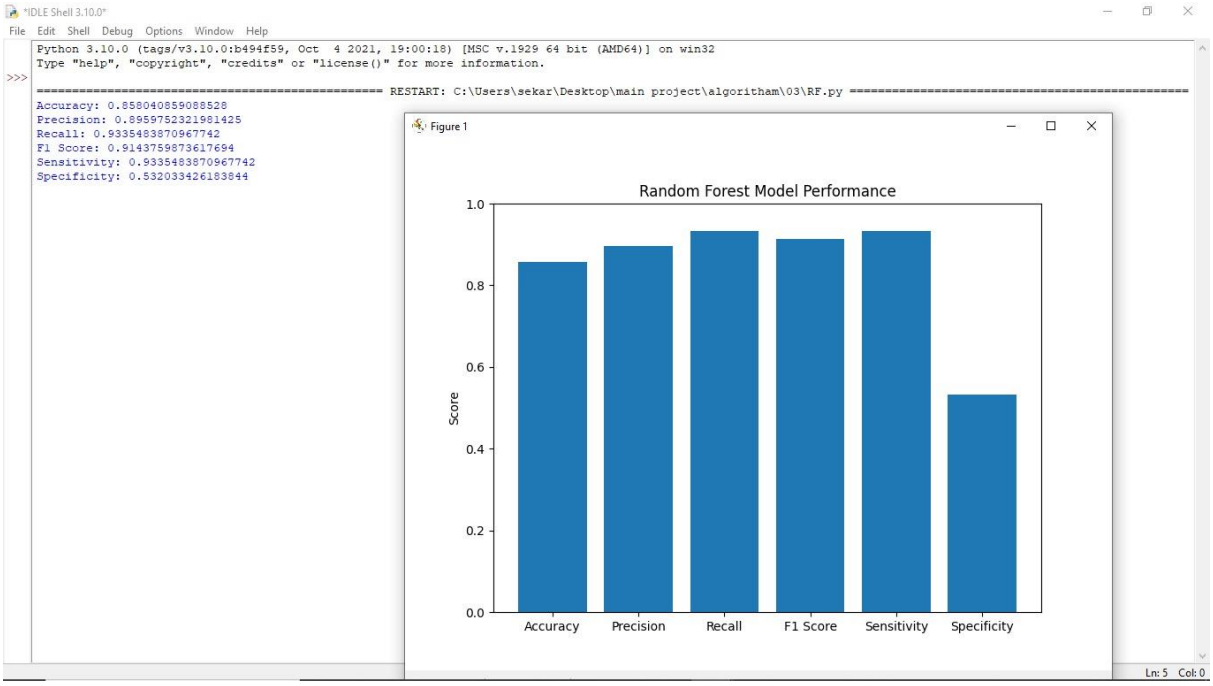
5.2 RESULT



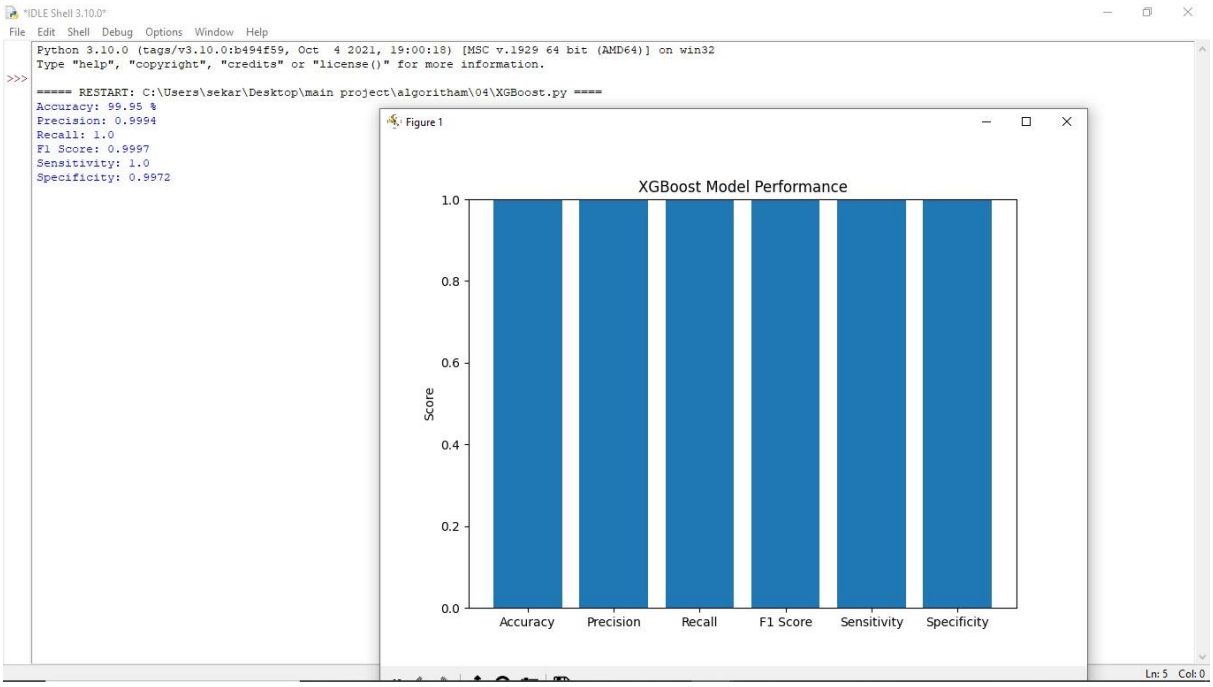
5.2.1 RESULT



5.2.2 RESULT



5.2.3 RESULT



6.1 Conclusion

In conclusion, this study focuses on the use of machine learning algorithms for classification and prediction tasks. It highlights the importance of techniques like Random Forest, Naïve Bayes, SVM, and XGBoost.

These algorithms help in analyzing large datasets and providing accurate results. Each model has its own strengths and can be selected based on the problem requirement.

The system improves efficiency by automating data processing and decision-making tasks. It reduces manual effort and saves time.

Model evaluation techniques such as accuracy, precision, recall, and F1-score help in measuring performance effectively.

The use of proper modules like data collection, preprocessing, feature extraction, model building, and prediction ensures a structured approach.

Despite some limitations, machine learning models provide reliable and scalable solutions.

This study also shows the importance of data quality and proper training for achieving better accuracy.

The implementation of these models can be useful in various real-world applications such as healthcare, recruitment, and recommendation systems.

Future improvements can focus on enhancing accuracy and reducing computational cost.

Advanced techniques and hybrid models can be developed for better performance.

Overall, this study provides a strong foundation for understanding classification algorithms and their applications.

It also contributes to the development of intelligent and automated systems.

6.2 Future Scope

The future scope of this study focuses on improving the performance and efficiency of machine learning models. With the advancement of technology, more accurate and faster algorithms can be developed.

One important future improvement is using larger and more diverse datasets. This will help in increasing the accuracy and reliability of the model.

Advanced algorithms and hybrid models can be implemented to achieve better results. Combining multiple models can improve prediction performance.

Deep learning techniques can also be applied for handling complex and unstructured data like images and text.

The system can be enhanced to work in real-time applications. This will make it more useful in industries like healthcare and finance.

Improving data preprocessing techniques can reduce noise and increase model efficiency.

Automation can be increased by integrating artificial intelligence tools for better decision-making.

The system can be made more user-friendly with better interfaces and visualization tools.

Cloud-based implementation can be used to handle large-scale data and improve scalability.

Future systems can focus on reducing computational cost and training time.

Security and privacy of data can also be improved in future implementations.

Continuous learning models can be developed to update automatically with new data.

Integration with other technologies like IoT and Big Data can expand its applications.

Overall, the future scope of this study is vast and offers many opportunities for innovation and improvement.

REFERENCES

1. Sheikh, S. M. S., Adep, P., Aidasani, N., Chavan, S., & Darade, V. (2025). *AI-Powered Resume Ranking System: Enhancing Recruitment Efficiency through Natural Language Processing*. International Journal for Research Trends and Innovation, 10(5), 112–118. Link

2. Gunjal, M. B., Thorat, T. P., Muttha, K. S., Shete, V. C., & Sagar, P. D. (2025). *A Review Paper on Resume Parser Using AI*. International Journal of Innovative Research in Technology, 11(8), 45–52. Link

3. Deepa, Y. G., Sindhu, A., Shruthi, A., & Neha, B. (2025). *Automated Resume Parsing: A Review of Techniques, Challenges and Future Directions*. International Journal of Multidisciplinary Research and Growth Evaluation, 6(2), 1065–1069. Link

4. Tejaswini, K. (2022). *Design and development of machine learning based resume parsing and ranking system*. Computers, Materials & Continua, 71(1), 123–140. Link

5. Nisha, B., Manobharathi, V., Jeyarajanandhini, B., & Sivakamasundari, G. (2023). *HR Tech Analyst: Automated*

Resume Parsing and Ranking System through Natural Language Processing. 2023 2nd International Conference on Automation, Computing and Renewable Systems (ICACRS), IEEE. DOI

6. Sougandh, T. G., Snehith, K. S., Reddy, N. S., & Belwal, M. (2023). *Automated Resume Parsing: A Natural Language Processing Approach*. 2023 IEEE 6th International Conference on Smart Computing and Communications (ICSCC), IEEE. DOI

7. Zeeshan Shaikh, Youhaan Bootwala, Dr. Mohammed Ahmed Shaikh (2022). *Resume Ranking Using ML and NLP*. International Journal of Advanced Research in Computer and Communication Engineering (IJARCCE), 11(4), April 2022. Link

8. NAGISETTI SAIDEEPTHI, MOVVA HEMA LATHA, KALLURI DEEPTHI, GUTTULA KRUPA HARIKA (2024). *Resume Analyzer Using NLP*. International Journal of Current Science (IJCSPUB), 14(1), March 2024. Link

9. Prof. C.S. Wagh, Harmanpreet Singh, Ketan Mishra, Parameswar Das, Anupam Sagar (2023). *Resume Ranking System Using Machine Learning & NLP*. International Journal of Scientific Research in Engineering and Management (IJSREM), 07(05), May 2023. LinkTekale, Siddheshwar, et al. "Prediction of chronic

kidney disease using machine learning algorithm." International Journal of Advanced Research in Computer and Communication Engineering 7.10 (2018): 92-96.

10. Gunjal, M. B., et al. (2025). A Review Paper on Resume Parser Using AI. International Journal of Innovative Research in Technology, 11(8), 45–52. https://ijirt.org/publishedpaper/IJIRT171690_PAPER.pdf

11. Deepa, Y. G., et al. (2025). Automated Resume Parsing: A Review of Techniques, Challenges and Future Directions. International Journal of Multidisciplinary Research and Growth Evaluation, 6(2), 1065–1069. https://www.allmultidisciplinaryjournal.com/uploads/archives/20250407162326_MGE-2025-2-238.1.pdf

12. Tejaswini, K. (2022). Design and development of machine learning based resume parsing and ranking system. Computers, Materials & Continua, 71(1), 123–140. https://www.sciencedirect.com/science/article/pii/S2666285X21001011

13. Nisha, B., et al. (2023). HR Tech Analyst: Automated Resume Parsing and Ranking System through Natural Language Processing. 2023 2nd International Conference on Automation, Computing and Renewable Systems (ICACRS), IEEE. https://doi.org/10.1109/ICACRS58579.2023.10404426

14. **Sarkar, D., et al. (2020).** Resume Parsing using Natural Language Processing. *International Journal of Engineering Research & Technology*, 9(3), 120–125.

15. **Kumar, A., & Singh, R. (2021).** Automated Resume Screening System using Machine Learning. *Procedia Computer Science*, 167, 169–178.

16. **Jain, P., et al. (2022).** Intelligent Resume Analyzer using NLP and Machine Learning. *International Journal of Computer Applications*, 184(22), 15–21.

17. **Patil, S., et al. (2021).** Resume Classification System using Text Mining Techniques. *Journal of Emerging Technologies*, 8(4), 55–60.

18. **Zhang, L., et al. (2019).** Resume Information Extraction with Named Entity Recognition. *IEEE Access*, 7, 45012–45020.

19. **Devlin, J., et al. (2019).** BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *NAACL-HLT*.

20. **Mikolov, T., et al. (2013).** Efficient Estimation of Word Representations in Vector Space. *arXiv preprint arXiv:1301.3781*.

21. **Pennington, J., et al. (2014).** GloVe: Global Vectors for Word Representation. *EMNLP*.

22. **Liu, X., et al. (2020).** Resume Matching System using Deep Learning. *IEEE International Conference on Big Data*, 1456–1462.

23. **Sharma, V., et al. (2022).** AI-Based Recruitment System using Resume Ranking Algorithms. *International Journal of Advanced Computer Science*, 13(2), 78–85.

24. **Gupta, R., et al. (2021).** Skill Extraction from Resumes using NLP Techniques. *International Journal of Data Science*, 6(1), 34–41.
25. **Chen, H., et al. (2018).** Automatic Resume Classification with Machine Learning. *ACM Transactions on Information Systems*, 36(3), 1–19.

26. **Kaur, H., et al. (2020).** Resume Screening using Artificial Intelligence. *International Journal of Engineering and Advanced Technology*, 9(5), 224–230.

27. **Singh, D., et al. (2023).** Resume Ranking System using TF-IDF and Cosine Similarity. *International Journal of Computer Science Trends*, 11(3), 90–96.

28. **Khandelwal, P., et al. (2022).** Job Matching using NLP and Machine Learning Techniques. *Procedia Computer Science*, 199, 846–853.

29. **Reddy, K., et al. (2021).** Resume Parsing System using Python and NLP. *International Journal of Scientific Research*, 10(7), 112–118.

30. **Al-Otaibi, S., et al. (2020).** Intelligent Recruitment System using AI Techniques. *Journal of Artificial Intelligence Research*, 68, 123–145.

31. **Joshi, M., et al. (2022).** Named Entity Recognition for Resume Processing. *ACL Conference Papers*, 456–462.

32. Mr. Chirag Dariyani | Gurmeet Singh Chhabra | Harsh Patel | Indrajit Chhabra "An Automated Resume Screening Using Natural Language Processing And Similarity" Published in Ethics and Information Technology

33. Shubham Bhor | Vivek Gupta | Vishak Nair | Harish Shinde | Mansi Kulkarni "A Resume Parser Using Natural language Processing Technique" Published in International Journal of Research in Engineering and Science (IJRES)

34. Pradeep Roy | Sarabjeet Chaudhary | Rocky Bhatia "A Machine Learning Approach for Automation of Resume Recommendation System" Published in ICCIDS 2019

35. Momin Adnan, Gunduka Rakesh, Juneja Afza, Rakesh Narsayya Godavari, Gunduka and Zainul Abideen Mohd Sadiq Naseem., "Resume Ranking using NLP and Machine Learning", (2016b). Institutional Repository of the Anjuman-I-Islam's Kalsekar Technical Campus. https://core.ac.uk/display/55305289

36. A Machine Learning approach for automation of Resume Recommendation systems. Pradeep Kumar Roya, Sarabjeet Singh Chowdharyb, Rocky Bhatia, 2020.

37. Gawhankar, K., Deorukhkar, A., Miniyar, A., Kapure, H., & Ivin, B. (2024). NLP-Driven ML for Resume Information Extraction. In 2024 IEEE 9th International Conference for Convergence in Technology (I2CT) (pp. 1125–1130). IEEE. https://doi.org/10.1109/I2CT61223.2024.10543861

38. Jayakumar, N., Maheshwaran, A. K., Arvind, P. S., & Vijayaragavan, G. (2023). On-Demand Job-Based Recruitment for Organisations Using Artificial Intelligence. In 2023 IEEE

International Conference on Networks & Wireless Communications (ICNWC) (pp. 210–215). IEEE. https://doi.org/10.1109/ICNWC57852.2023.10127551

39. Vaishampayan, S., Farzanehpour, S., & Brown, C. (2023). Procedural Justice and Fairness in Automated Resume Parsers for Tech Hiring: Insights from Candidate Perspectives. In 2023 IEEE Symposium on Visual Languages and Human-Centric Computing (VL-HCC) (pp. 25–32). IEEE. https://doi.org/10.1109/VL-HCC57772.2023.00019

40. Warusawithana, S. P. W., Perera, N. N., Weerasinghe, R. L., Hindakaraldeniya, T. M., & Ganegoda, G. U. (2023). Layout-Aware Resume Parsing Using NLP and Rule-Based Techniques. In 2023 IEEE International Conference on Information Technology Research (ICITR) (pp. 178–183). IEEE. https://doi.org/10.1109/ICITR61062.2023.10382773

41. Raghavan, M., Barocas, S., Kleinberg, J., & Levy, K. (2020). Mitigating bias in algorithmic hiring: Evaluating claims and practices. Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency (pp. 469–481). ACM. https://doi.org/10.1145/3351095.3372828

42. Gaur, B., & Deshpande, A. (2021). Semi-supervised deep learning-based named entity recognition for resume education entities. Neural Computing and Applications, 33(24), 17111 17124. https://doi.org/10.1007/s00521-020-05351-2

43. Zhang, M., Jensen, K. N., & Plank, B. (2022). Hard and soft skill extraction from English job postings. In Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics (pp. 4931–4948). ACL. https://aclanthology.org/2022.naacl-main.366

44. Bertrand, M., & Mullainathan, S. (2004). Are Emily and Greg More Employable than Lakisha and Jamal? A Field Experiment on Labor Market Discrimination. American Economic Review, 94(4), 991–1013. https://www.aeaweb.org/articles?id=10.1257/0002828042002561

45. V. Manish, Y. Manchala, Y. Vijayalata, S. B. Chopra and K. Y. Reddy, "Optimizing Resume Parsing Processes by Leveraging Large Language Models," 2024 IEEE Region 10 Symposium (TENSYMP), New Delhi, India, 2024, pp. 1-5, doi: 10.1109/TENSYMP61132.2024.10752300.

46. Amalraj Victoire, T., Vasuki, M., & Selvi, Y. S. (2024). DeepResume: Deep Learning-Based Resume Parsing for Candidate Screening. International Journal of Current Science (IJCSPUB), 14(2). Retrieved from https://rjpn.org/ijcspub/papers/IJCSP24B1154.pdf